

22. Poser une tuile

Une lettre à l'écran est une tuile comme une autre.

1. Ce qu'il faut comprendre

Les deux lignes affichent la même chose. `texte()` traduit des lettres en numéros de tuiles ; `poser(colonne, ligne, tuile)` pose le numéro directement.

La tuile 1 est le `A` de la police fournie, la 2 est un `B`. Il n'y a pas de magie : dessiner sa propre tuile, ce sera **ajouter un numéro à cette même suite**.

`lire(colonne, ligne)` fait l'inverse — il rend le numéro affiché à cet endroit. C'est ce qui servira pour les collisions.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1401 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(2, 4, "AVEC TEXTE ABC");

    texte(2, 8, "AVEC POSER");
    poser(14, 8, 1);
    poser(15, 8, 2);
    poser(16, 8, 3);

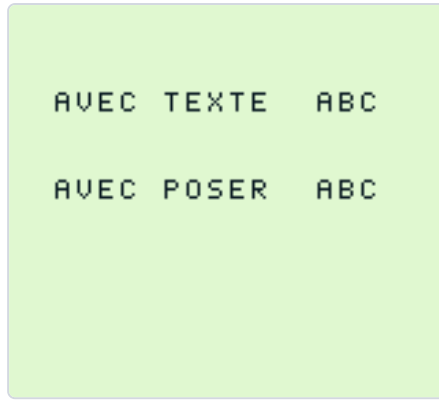
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le même « ABC » sur les deux lignes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ `texte()` écrit ABC
- ✓ `poser()` écrit le même ABC
- ✓ ce sont bien les mêmes numéros
- ✓ `lire()` les retrouve

Ensuite : **23. Quatre nuances, quatre signes** — niveau 3 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++